

第24回 仲間が増える・敵が消える:ArrayList

今回のゴール

- `ArrayList` を使って、途中で人数が変わるパーティーと敵のグループを管理できる。
- `add`・`get`・`remove`・`size`・`isEmpty` を、必要な場面で使える。
- 配列と `ArrayList` の違いを説明できる。
- `import` が何のために必要か説明できる。

前回とのつながり

第23回までのパーティーは配列 `Character[]` だった。配列は、最初に決めた要素の数を後から変えられない。そのため、

- 戦闘の途中で仲間が加わる
- たおした敵をグループから取り除く
- たおれた仲間を攻撃の順番から外す

ことが難しかった。今回は、要素の数を自由に変えられる `ArrayList` を使って、敵を複数にする。

実習時のフォルダ構成

```
ワークスペース
├─ java24_sample
│   └─ src
│       ├─ Character.java
│       ├─ Hero.java
│       ├─ Wizard.java
│       ├─ Warrior.java
│       ├─ Priest.java
│       ├─ Enemy.java
│       └─ Main.java
└─ java24_exercise
    └─ src
        └─ (sample と同じ 7 ファイル)
```

- 配布ファイル: `プログラム/第24回/sample/`、`プログラム/第24回/exercise/`
- 解答例: `プログラム/第24回/answer/`
- `Main.java` 以外のクラスは、第23回の解答例と同じ。

1. ArrayList の基本

```
import java.util.ArrayList; // ファイルの一番上を書く

ArrayList<Character> party = new ArrayList<>(); // 空のリストを作る
party.add(hero); // 最後に追加
Character first = party.get(0); // 0 番目を取り出す
party.remove(hero); // hero を取り除く
int count = party.size(); // 要素の数
boolean none = party.isEmpty(); // 空なら true
```

操作	配列	ArrayList
作る	<code>Character[] a = new Character[3];</code>	<code>ArrayList<Character> list = new ArrayList<>();</code>
要素の数	<code>a.length</code> (変えられない)	<code>list.size()</code> (増えたり減ったりする)
取り出す	<code>a[0]</code>	<code>list.get(0)</code>
追加	できない	<code>list.add(x)</code>
取り除く	できない	<code>list.remove(x)</code> または <code>list.remove(番号)</code>

- `<Character>` は「このリストには `Character` を入れる」という指定。`ArrayList<Enemy>` なら `Enemy` だけ入れられる。
- 右側の `new ArrayList<>()` の `<>` の中は、左側から分かるので省略できる。
- 拡張 `for` 文は、配列と同じように使える。
- `remove` した後は、後ろの要素が 1 つずつ前につめられる。

2. import

`ArrayList` や `Scanner` は、Java に最初から用意されている部品(API)だが、`java.util` というパッケージ(部品のグループ)に入っている。使うときは、ファイルの先頭に `import` を書いて「`java.util` の `ArrayList` を使う」と宣言する。

```
import java.util.ArrayList;
import java.util.Scanner;
```

- `String` や `System` は `java.lang` というパッケージにある。`java.lang` だけは特別に、`import` を書かなくても使える。
- 第17回から書いていた `import java.util.Scanner;` も、これと同じ意味だった。

3. サンプルを読む

プログラム/第24回/sample/Main.java

```
import java.util.ArrayList;
```

```

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String[] commands = { "攻撃", "薬草を使う", "にげる", "ためる" };

        Hero hero = new Hero("勇者", 30, 8, 2);
        Priest priest = new Priest("僧侶", 25, 4);

        // 味方パーティー（人数が変わるので ArrayList を使う）
        ArrayList<Character> party = new ArrayList<>();
        party.add(hero);
        party.add(new Wizard("魔法使い", 15, 5));
        party.add(new Warrior("戦士", 40, 10));

        // 敵のグループ
        ArrayList<Enemy> enemies = new ArrayList<>();
        enemies.add(new Enemy("スライム", 20, 5, 5));
        enemies.add(new Enemy("ゴブリン", 40, 7, 12));
        enemies.add(new Enemy("オーク", 70, 9, 30));

        int turn = 1;
        int totalExp = 0;
        boolean escaped = false;

        System.out.println("魔物の群れがあらわれた！");

        while (!party.isEmpty() && !enemies.isEmpty()) {
            System.out.println();
            System.out.println("--- ターン" + turn + " ---");

            // 2 ターン目に、僧侶が仲間に加わる
            if (turn == 2) {
                party.add(priest);
                System.out.println(priest.getName() + "が仲間に加わった！");
            }

            for (Character member : party) {
                member.showStatus();
            }
            System.out.println("敵：" + enemies.size() + "体");
            for (Enemy enemy : enemies) {
                enemy.showStatus();
            }

            int command = inputCommand(scanner, commands);

            switch (command) {
                case 1:
                    // 全員が、先頭の敵を攻撃する
                    for (Character member : party) {
                        if (enemies.isEmpty()) {
                            break;
                        }
                        Enemy target = enemies.get(0);

```

```

        member.attack(target, turn);
        if (!target.isAlive()) {
            System.out.println(target.getName() + "をたおした!");
            totalExp += target.getExp();
            enemies.remove(target);
        }
    }
    break;
case 2:
    if (hero.isAlive()) {
        hero.useHerb();
    } else {
        System.out.println(hero.getName() + "はたおれていて薬草を使えない!");
    }
    break;
case 3:
    System.out.println("パーティーはにげだした!");
    escaped = true;
    break;
case 4:
    hero.powerUp(3);
    System.out.println(hero.getName() + "は力をためた! 攻撃力が" + hero.getPower() +
        "になった");
    break;
}

if (escaped) {
    break;
}

// 残っている敵が、1 体ずつ攻撃する
for (Enemy enemy : enemies) {
    if (party.isEmpty()) {
        break;
    }
    Character target = party.get(turn % party.size());
    enemy.attack(target, turn);
    if (!target.isAlive()) {
        System.out.println(target.getName() + "はたおれた...");
        party.remove(target);
    }
}
turn++;
}

System.out.println();
if (escaped) {
    System.out.println("うまくにげきれた。");
} else if (enemies.isEmpty()) {
    System.out.println("魔物の群れをたおした! 経験値" + totalExp + "を手に入れた");
} else {
    System.out.println("パーティーは全滅した...");
}
}

// inputCommand は第17回から同じ (省略)

```

```
}
```

- たおれた仲間・たおした敵は、リストから取り除く。だから `isAlive()` を調べなくても、「リストにいる＝戦える」になる。
- 全滅の判定は、第22回の `isPartyAlive` の代わりに `party.isEmpty()` で済むようになった。
- `for` の中の `break` は、その `for` 文だけを抜ける。敵が全滅したら、残りのメンバーは攻撃しない。
- 敵がねらう相手は、第22回と同じく `turn % party.size()` で順番に決めている。人数が変わっても、`size()` が変わるので範囲外にならない。
- 注意: 拡張 `for` 文で **くり返している最中のリスト** から `remove` すると、実行時エラーになる。サンプルでは、`party` をくり返しながら `enemies` から取り除き、`enemies` をくり返しながら `party` から取り除いているので問題ない。

4. 実行する前に予想する

1 を続けて入力する。

- 1 ターン目の終わりに、誰がたおれるか(敵 2 体が同じ相手をねらう)。
- 2 ターン目の最初に、パーティーは何人になるか。

5. 実行結果(1～2 ターン目)

魔物の群れがあらわれた！

--- ターン1 ---

勇者 HP:30/30

魔法使い HP:15/15

戦士 HP:40/40

敵：3体

スライム HP:20/20

ゴブリン HP:40/40

オーク HP:70/70

1: 攻撃

2: 薬草を使う

3: にげる

4: ためる

コマンドを選ぶ > 1

勇者の攻撃！ スライムに8のダメージ

魔法使いは火の魔法をとねた！ スライムに10のダメージ

戦士のおのの一撃！ スライムに10のダメージ

スライムをたおした！

ゴブリンの攻撃！ 魔法使いに7のダメージ

オークの攻撃！ 魔法使いに9のダメージ

魔法使いはたおれた...

--- ターン2 ---

僧侶が仲間に加わった！

勇者 HP:30/30

戦士 HP:40/40

僧侶 HP:25/25

敵：2体

ゴブリン HP:40/40

オーク HP:70/70

最後まで実行すると、6 ターン目に次のように終わる。

会心の一撃！

勇者の攻撃！ オークに12のダメージ

戦士のおのの一撃！ オークに10のダメージ

オークをたおした！

魔物の群れをたおした！ 経験値47を手に入れた

6. エラーを体験する

1. `import java.util.ArrayList;` の行を消して保存する。
2. `ArrayList` が見つからない(型に解決できない)というエラーが、`ArrayList` を使っているすべての行に出ることを確認する。
3. `import` を元に戻す。Pleiades では、`Ctrl` + `Shift` + `0` で必要な `import` を自動で追加することもできる。

演習1 サンプルを実行する

`java24_sample` でサンプルを実行し、予想と結果を比べる。

演習2 リストを操作する

`java24_exercise` を作り、`プログラム/第24回/exercise/` の 7 ファイルを入れる。

演習2-1 敵を追加する

- まず、そのまま実行する。すぐに「魔物の群れをたおした！」と表示されて終わる理由を考える。
- スライム(HP 20、攻撃力 5、経験値 5)、ゴブリン(40、7、12)、オーク(70、9、30)の 3 体を、この順に `enemies` に追加する。

演習2-2 たおれた仲間を外す

- 演習2-1の後で実行すると、HP が 0 になった魔法使いが、次のターンも表示され、攻撃している。
- 敵の攻撃の後で、`target` がたおれていたら `〇〇はたおれた...` と表示し、`party` から取り除く。

演習2-3 敵の増援

- 4 ターン目の最初に、`enemies` の最後にスライム(20、5、5)を追加し、`スライムがあらわれた！` と表示する。

ヒント:僧侶が仲間に加わる処理(`if (turn == 2)`)と同じ形で書ける。

完成条件(4 ターン目の最初):

```
--- ターン4 ---
スライムがあらわれた！
勇者 HP:21/30
戦士 HP:40/40
僧侶 HP:9/25
敵:2体
オーク HP:66/70
スライム HP:20/20
```

発展(任意)

- たおした敵の名前を `ArrayList<String>` に記録しておき、戦闘の最後に一覧で表示する。
- 「にげる」を選んだとき、敵が 1 体だけならにげられないようにする。

確認問題

問1 出力は何か。

```
ArrayList<String> items = new ArrayList<>();
items.add("薬草");
items.add("毒消し");
items.add("聖水");
items.remove(0);
System.out.println(items.size() + " " + items.get(0));
```

問2 次のコードを実行すると、どうなるか。

```
ArrayList<String> list = new ArrayList<>();
list.add("A");
list.add("B");
System.out.println(list.get(2));
```

問3 `import java.util.ArrayList;` を書かずに `ArrayList` を使うと、どうなるか。また、`String` は `import` なしで使えるのはなぜか。

■ 答え

- 問1: 2 毒消し (0 番目の「薬草」が取り除かれ、後ろが詰められる)
- 問2: 実行時エラー (`IndexOutOfBoundsException`)。要素は 0 番と 1 番の 2 つしかない
- 問3: コンパイルエラーになる。`String` は `java.lang` パッケージのクラスで、`java.lang` は自動で使えるようになっているから

振り返り

- 配列ではなく `ArrayList` を使った理由を、バトルの例で説明できるか。
- `add`・`get`・`remove`・`size` を使えたか。
- `import` が必要なクラスと、必要ないクラスの違いを説明できるか。

次回へのつながり

クラスのファイルが 7 つに増えた。次回は、バトル用のクラスを `パッケージ` にまとめて整理する。また、敵の行動をランダム(乱数)にして、毎回違うバトルにする。